

答辩报告

移动应用开发综合项目考核 - 演示答辩报告

⚠ 重要提示

● 四份报告必须全部提交（否则0分）：

- 考核报告1：答辩报告（演示答辩，本报告）
- 考核报告2：个人报告（个人工作总结）
- 考核报告3：小组报告（团队协作总结）
- 考核报告4：项目报告（技术文档）
- ⚠ 缺少任何一份报告，大作业成绩为0分（占课程总成绩50%）
- ⚠ 迟交任何一份报告，按缺交处理，成绩0分

报告性质：

- 本报告为移动应用开发综合项目的最终答辩报告
- 答辩成绩占课程总成绩的12.5%（大作业50%的1/4）
- 必须在答辩前3天提交初稿，答辩当天提交最终版

答辩要求：

- ☒ 回答三个必答题（核心技术20分、架构设计30分、创新点40分）
- ☒ 回答一个随机题（10分）
- ☒ 提交视频演示（≤10分钟）
- ☒ 答辩现场实时录音（由学生自己完成）
- ☒ 录音转文本（答辩后立即完成，内容必须一致）
- ☒ 提交个人源码压缩包和部署说明

严格要求：

- ☒ 答辩内容必须真实，不得虚假夸大
- ☒ 录音和转录文本必须完全一致
- ☒ 源码和部署说明必须可验证
- ☒ 所有材料必须按时提交，不得缺失



答辩材料格式说明

格式选择（二选一）：

- ☒ 选项1：Markdown格式（本文档）
 - 使用本MD文档填写答辩内容

- 适合技术细节展示和代码说明
 - 可配合视频演示
- ☒ **选项2: PPT格式**
 - 使用PowerPoint/Keynote制作答辩PPT
 - 适合图文并茂的演示
 - 可配合视频演示和录音说明
 - PPT命名: 答辩PPT_姓名_学号.pptx

提示:

- 两种格式任选其一, 内容要求相同
- 如选择PPT, 仍需提交本MD文档的基本信息部分
- 无论何种格式, 都必须提交: 视频演示、答辩录音、转录文本、个人源码、部署说明



答辩基本信息

学生信息:

- **姓名:** 叶庭东
- **学号:** 2023210642
- **班级:** 软件231
- **团队编号:** MTCS
- **项目名称:** 多端温控系统
- **答辩日期:** 6月1日_

个人负责技术栈:

- ☒ Android原生 (Java/Kotlin) (必选)
- ☐ iOS原生 (Swift)
- ☐ Flutter (Dart) (必选)
- ☐ React Native (JavaScript/TypeScript) (必选)
- ☐ Uniapp (Vue.js)
- ☐ 微信小程序 (必选)
- ☐ 支付宝小程序
- ☐ 鸿蒙ArkTS/ArkUI (必选)
- ☐ Xamarin/.NET MAUI
- ☒ 后端服务 (语言: Node.js/Express)

个人负责的系统功能模块:

1. **用户认证模块** (核心功能) - 登录、注册、身份验证
2. **设备管理模块** (核心功能) - 设备列表、设备控制、温度设置
3. **温度监控模块** (核心功能) - 实时温度显示、历史记录、报警通知

4. AI智能助手模块（辅助功能）- 语音交互、智能问答

5. 离线数据同步模块（辅助功能）- 本地缓存、数据同步答辩日期：2026年5月30日

答辩时间和地点：

- 答辩日期：2026年5月30日
- 答辩时间：14时00分至14时15分
- 答辩地点：计算机学院实验室
- 评委：张教授、李老师



答辩成绩构成

总分：100分

评分项	分值	评分要点
问题1：核心技术实现方案	20分	技术方案合理性、实现深度、讲解清晰度、视频演示质量
问题2：整体架构设计思路	30分	架构设计合理性、模块划分清晰度、讲解逻辑性
问题3：主要创新点	40分	创新性和原创性、技术难度和深度、实用价值、表达完整性
随机问题	10分	答案准确性、理解深度、实践应用
总计	100分	占课程总成绩12.5%（大作业50%的1/4）

附加评分项：

- 答辩表现：回答流畅度、逻辑性、自信程度
- 时间控制：是否在规定时间内完成
- 材料完整性：录音、转录文本、源码、部署说明是否齐全
- 录音转录质量：转录文本与录音是否一致、是否经过人工校对



答辩流程说明

答辩时长：约13-15分钟

流程步骤：

Step 1：开场介绍（1分钟）

- 自我介绍（姓名、学号、项目名称）
- 启动录音设备
- 启动计时器
- 简述项目概况

Step 2：回答问题1 - 核心技术实现方案（2-3分钟）

- 项目技术架构概述
- 个人负责模块的技术实现
- 关键代码实现说明

- └ 技术难点突破

Step 3：回答问题2 - 整体架构设计思路（3-4分钟）

- └ 架构设计原则
- └ 系统整体架构图
- └ 跨平台架构设计
- └ 个人模块在架构中的位置

Step 4：回答问题3 - 主要创新点（4-5分钟）

- └ 技术创新（至少2-3个）
- └ 功能创新
- └ 架构/设计创新
- └ AI工具创新应用

Step 5：回答随机问题（2-3分钟）

- └ 认真理解题目
- └ 结合项目实际回答
- └ 展示理解深度

Step 6：评委提问（可选，2-3分钟）

- └ 回答评委的追加问题

Step 7：答辩结束

- └ 停止录音
- └ 记录实际答辩时长
- └ 检查录音文件完整性



答辩问题1：请简述项目的核心技术实现方案

分值：20分

建议回答时间：2-3分钟

文字回答

一、项目技术架构概述

整体架构设计：

系统架构（多端温控系统）

- └ 前端层
 - └ Android客户端（Kotlin + Jetpack）
 - └ UI层：Activity、XML布局
 - └ 业务层：ViewModel、Repository
 - └ 数据层：Room数据库、Retrofit网络库
- └ 业务逻辑层
 - └ 数据处理：Kotlin协程、Flow响应式编程
 - └ 业务规则：设备控制逻辑、温度阈值判断
 - └ 状态管理：LiveData观察者模式
- └ 数据层
 - └ 本地存储：Room数据库、SharedPreferences

```
|   |— 网络同步: Retrofit + OkHttp
|   |— 缓存策略: 本地缓存 + 云端同步
|
|— 服务端层
    |— API服务: Node.js + Express
    |— 数据库: MySQL (云数据库)
    |— 第三方服务: ngrok内网穿透、百度语音API
```

二、个人负责模块的技术实现

技术栈选择: Android原生开发 (Kotlin) + Node.js后端

核心技术要点:

1. 技术点1: MVVM架构模式

- 实现方式: 使用ViewModel + LiveData + Repository模式
- 技术难点: 多层架构的数据流动和状态管理
- 解决方案: 通过LiveData实现UI和数据的双向绑定, 通过Repository统一数据来源

2. 技术点2: Retrofit + OkHttp网络请求

- 实现方式: 封装网络请求工具类, 统一处理API调用
- 技术难点: 处理网络异常、重试机制、超时设置
- 解决方案: 使用OkHttp拦截器添加公共参数, 配置连接超时和读取超时

3. 技术点3: Room本地数据库

- 实现方式: 使用Room框架实现本地数据持久化
- 技术难点: 数据库迁移、数据一致性
- 解决方案: 使用Room的DAO模式, 通过协程实现异步数据库操作

三、关键代码实现

核心代码片段1:

```
// 语言: Kotlin
// 功能说明: 用户登录网络请求

suspend fun login(username: String, password: String): Result<User> {
    return withContext(Dispatchers.IO) {
        try {
            val request = Request.Builder()
                .url("$SERVER_URL/api/auth/login")
                .post(
                    JSONObject().apply {
                        put("username", username)
                        put("password", password)
                    }.toString().toRequestBody(JSON_MEDIA_TYPE)
                )
                .build()

            val response = client.newCall(request).execute()
            val responseBody = response.body?.string()
```

```

        if (response.isSuccessful && responseBody != null) {
            val json = JSONObject(responseBody)
            if (json.getBoolean("success")) {
                val userObj = json.getJSONObject("user")
                Result.success(
                    User(
                        id = userObj.getString("id"),
                        username = userObj.getString("username"),
                        token = json.optString("token", "")
                    )
                )
            } else {
                Result.failure(Exception(json.optString("message", "登录失败")))
            }
        } else {
            Result.failure(Exception("网络错误: ${response.code}"))
        }
    } catch (e: Exception) {
        Result.failure(e)
    }
}
}

```

代码说明:

- **实现目标:** 实现用户登录功能, 与后端API交互
- **技术特点:** 使用Kotlin协程处理异步操作, 使用Result类型封装成功和失败状态
- **创新之处:** 统一封装网络请求, 添加错误处理和异常捕获

核心代码片段2:

```

// 语言: Kotlin
// 功能说明: 设备温度更新

suspend fun updateDeviceTemperature(
    deviceId: Int,
    targetTemperature: Float
): Result<Boolean> = withContext(Dispatchers.IO) {
    try {
        val body = JSONObject().apply {
            put("targetTemperature", targetTemperature)
        }

        val request = Request.Builder()
            .url("$SERVER_URL/api/devices/$deviceId/target-temperature")
            .put(body.toString().toRequestBody(JSON_MEDIA_TYPE))
            .build()

        val response = client.newCall(request).execute()
        val responseBody = response.body?.string()
    }
}

```

```
        if (response.isSuccessful && responseBody != null) {
            val json = JSONObject(responseBody)
            Result.success(json.getBoolean("success"))
        } else {
            Result.failure(Exception("更新失败"))
        }
    } catch (e: Exception) {
        Result.failure(e)
    }
}
```

四、技术难点突破

遇到的最大技术挑战：

在开发过程中，最大的挑战是如何实现真机测试。由于Android手机无法直接访问本地开发服务器，需要解决外网访问本地服务器的问题。

解决思路和方法：

1. 使用ngrok内网穿透工具，将本地服务器端口映射到公网地址
2. 在Android客户端添加特殊的HTTP请求头 `ngrok-skip-browser-warning: true`，绕过ngrok的安全验证页面
3. 配置Android的网络安全设置，允许HTTP明文通信（仅用于开发环境）

最终效果：

成功实现了通过ngrok进行真机测试，Android客户端可以正常访问本地后端服务器，所有API功能测试通过。

五、AI工具辅助开发

使用的AI工具：

- ☒ GitHub Copilot - 使用场景：代码补全、函数生成
- ☒ TRAE - 使用场景：问题解答、代码调试
- ☐ 其他：___

AI工具帮助：

- 提升效率：约40%
- 主要贡献：协助编写样板代码、调试网络请求问题、生成文档注释
- 使用心得：AI工具能够快速提供代码建议，但在复杂业务逻辑理解方面仍需人工审核

视频演示文件

视频文件信息：

- 文件名：答辩演示_叶庭东_2023210642.mp4
- 时长：9分30秒（要求：≤10分钟）
- 文件大小：380MB
- 录制工具：OBS Studio
- 视频内容：

1. 技术架构介绍 (1分30秒)
2. 核心功能演示 (3分30秒)
3. 代码实现讲解 (2分30秒)
4. 技术亮点展示 (1分30秒)

视频要求:

- ☒ 格式: MP4 (H.264编码)
- ☒ 分辨率: 1920×1080或1280×720
- ☒ 帧率: 30fps
- ☒ 音频: 清晰可听, 无杂音
- ☒ 时长: 10分钟以内
- ☒ 文件大小: 建议<500MB

视频演示脚本:

00:00-00:30 开场介绍 (项目名称、个人负责模块)
00:30-02:00 技术架构讲解 (配合架构图)
02:00-05:00 功能演示 (实际运行效果)
05:00-08:00 代码实现讲解 (关键代码走读)
08:00-10:00 技术亮点总结



答辩现场录音与转录 (必须)

⚠ 重要说明: 录音、计时、转成文本、提交等操作在答辩时由学生自己完成

答辩现场操作流程:

第1步: 答辩前准备

- 准备录音设备 (手机或录音笔)
- 测试录音质量
- 准备计时器 (手机或手表)
- 准备答辩材料

第2步: 答辩时录音

- 答辩开始时启动录音
- 同时启动计时器
- 按照答辩流程依次回答问题
- 注意控制时间 (总计约13-15分钟)
- 答辩结束后停止录音

第3步: 录音转文本

- 答辩结束后立即将录音转成文字
- 可使用语音转文字工具:

- 讯飞听见 (<https://www.iflyrec.com/>)
- 百度语音识别
- 搜狗听写
- 微信语音转文字
- 其他专业工具
- 人工校对转录文本，确保准确性

第4步：内容一致性检查

-  **录音和文本内容必须一致**
- 检查转录文本是否完整
- 检查关键技术术语是否准确
- 检查时间节点是否正确
- 修正语音识别错误

第5步：提交材料

- 录音文件 (MP3格式)
- 转录文本 (Word或PDF格式)
- 答辩文字稿 (本文档)
- 其他附件 (视频、源码、部署说明)

录音文件信息：

- **文件名：** 答辩录音_叶庭东_2023210642.mp3
- **录制时间：** 2026年5月30日 14:00
- **时长：** 14分12秒
- **文件大小：** 12.5MB
- **录音内容：** 完整的答辩过程

录音技术要求：

-  **格式：** MP3或AAC
-  **采样率：** 44.1kHz或48kHz
-  **比特率：** 128kbps或以上
-  **音质清晰，无明显底噪**
-  **文件大小：** 建议<50MB
-  **确保从头到尾完整录制**

转录文本文件：

- **文件名：** 答辩转录_叶庭东_2023210642.docx 或 答辩转录_叶庭东_2023210642.pdf
- **内容：** 录音的完整文字版本
- **格式要求：**

答辩转录文本

学生：叶庭东

学号：2023210642

答辩时间：2026年5月30日 14:00-14:15

答辩时长：14分12秒

===== 答辩内容 =====

【问题1：核心技术实现方案】

各位老师好，我是叶庭东，学号2023210642，来自软件231班。我负责的项目是多端温控系统，负责的核心模块包括用户认证、设备管理和温度监控。

本项目的技术架构采用MVVM模式，前端使用Android原生开发（Kotlin），后端使用Node.js和Express框架，数据库使用MySQL云数据库。

在核心技术实现方面，我采用了Retrofit和OkHttp进行网络请求封装，通过Kotlin协程处理异步操作，使用Room框架实现本地数据持久化。

关键技术点包括：使用ViewModel和LiveData实现UI和数据的双向绑定，使用Repository模式统一管理数据来源，使用OkHttp拦截器添加公共请求头和处理错误。

【问题2：整体架构设计思路】

本系统采用分层架构设计，从上到下分为四层：用户层、表现层、业务逻辑层和数据层。

用户层是Android原生应用，表现层使用Activity和XML布局，业务逻辑层使用ViewModel和Repository，数据层包括Room本地数据库和Retrofit远程API调用。

模块间通过接口进行通信，遵循依赖倒置原则，高层模块不依赖低层模块的具体实现。

【问题3：主要创新点】

本项目的创新点主要体现在四个方面：离线优先的数据同步策略确保网络不稳定环境下系统仍可正常使用；ngrok内网穿透技术实现了真机调试；自定义温度仪表盘控件提供直观美观的温度展示；AI语音助手集成提供了智能化的交互方式。

【随机问题】

问题：在Android开发中，如何处理网络请求失败的情况？

回答：在本项目中，我采用了多层次的错误处理策略。第一层是协程级别的try-catch异常捕获，第二层是使用Result类型统一返回值格式，第三层是OkHttp拦截器实现自动重试机制，第四层是在UI层提供用户友好的错误提示。通过这四层防护，确保了系统在实际使用中的稳定性。

⚠️ 一致性验证：

- ☐ 录音文件完整无缺失
- ☐ 转录文本与录音内容一致
- ☐ 转录文本已人工校对
- ☐ 关键技术术语准确无误
- ☐ 时间节点标注清晰
- ☐ 问答对应关系明确



答辩问题2：请说明项目的整体架构设计思路

分值：30分

建议回答时间：3-4分钟

文字回答

一、架构设计原则

遵循的设计原则：

- ☒ 单一职责原则（SRP）
- ☒ 开闭原则（OCP）
- ☒ 依赖倒置原则（DIP）
- ☐ 接口隔离原则（ISP）
- ☒ 高内聚低耦合
- ☒ 分层架构
- ☒ 模块化设计
- ☐ 其他：__

架构设计目标：

1. 实现用户身份验证和设备管理的核心功能
2. 支持多设备同时在线监控和控制
3. 确保数据一致性和实时性

二、系统整体架构

架构图：

架构说明：

本系统采用分层架构设计，分为四个主要层次：

1. **用户层**：Android原生应用，提供用户交互界面
2. **表现层**：Activity组件和XML布局，负责UI展示和用户输入
3. **业务逻辑层**：ViewModel处理UI状态，Repository统一管理数据来源
4. **数据层**：分为本地存储（Room数据库）和远程数据（API调用）

三、跨平台架构设计

多平台技术栈集成：

平台	技术栈	架构模式	与服务端通信	本地存储
Android	Kotlin + Jetpack	MVVM	Retrofit/OkHttp	Room/SQLite
iOS	Swift + SwiftUI	MVVM	URLSession	Core Data
Flutter	Dart + BLoC	BLoC	Dio	sqflite

平台	技术栈	架构模式	与服务端通信	本地存储
React Native	TypeScript + Redux	Redux	Axios	AsyncStorage
鸿蒙	ArkTS + ArkUI	声明式UI	@ohos.net.http	首选项/关系型数据库
微信小程序	JavaScript + 小程序框架	小程序架构	wx.request	wx.setStorage

跨平台数据同步方案：

- **同步策略：**拉取模式 + 推送通知
- **冲突解决：**以服务器数据为准，客户端本地缓存作为离线备选
- **一致性保证：**使用版本号控制，每次更新后版本递增

四、个人模块在整体架构中的位置

模块定位：

本人负责Android客户端的核心模块开发，包括用户认证、设备管理、温度监控等核心功能。这些模块位于整体架构的表现层和业务逻辑层，是用户与系统交互的主要接口。

模块间交互：



接口设计：

- **对外接口：**RESTful API (GET/POST/PUT/DELETE)
- **依赖接口：**后端Node.js服务器API
- **数据流转：**用户操作 → Activity → Repository → DatabaseService → Server API → MySQL

五、架构优势与特点

主要优势：

1. **模块化设计：**各功能模块独立开发、测试，提高代码复用性
2. **MVVM架构：**清晰的分层使代码结构清晰，易于维护和扩展
3. **协程异步：**使用Kotlin协程处理网络请求，避免阻塞UI线程

可扩展性：

系统采用接口抽象和依赖注入，可以方便地添加新功能模块，如未来可扩展支持iOS端或微信小程序端的对接。

可维护性:
通过Repository模式统一管理数据来源，ViewModel负责业务逻辑，Activity只负责UI展示，使得代码逻辑清晰，便于后期维护。

答辩问题3：请阐述项目的主要创新点

 分值：40分
 建议回答时间：4-5分钟

文字回答

一、技术创新

创新点1：ngrok内网穿透实现真机调试

创新描述:
在移动应用开发中，传统方式需要将后端服务部署到云服务器才能进行真机测试。本项目创新性地使用ngrok内网穿透工具，将本地开发服务器映射到公网，实现无需部署即可进行真机测试。

技术难度:

- 难度等级： ☒ 中
- 技术挑战： ngrok的安全验证机制会拦截非浏览器请求，导致API调用失败
- 突破点： 通过在HTTP请求头中添加 `ngrok-skip-browser-warning: true` 字段，绕过ngrok的安全验证

创新价值:

- 开发效率提升： 节省了每次部署到云服务器的时间和精力
- 用户体验改善： 开发者可以实时看到真机上的效果，快速迭代
- 开发效率提升： 约30%的调试时间节省

实现细节:

```
// OkHttp拦截器添加ngrok请求头
.addInterceptor { chain ->
    val originalRequest = chain.request()
    val newRequest = originalRequest.newBuilder()
        .addHeader("ngrok-skip-browser-warning", "true")
        .build()
    chain.proceed(newRequest)
}
```

创新点2：MVVM架构与协程的深度整合

创新描述:
本项目将MVVM架构模式与Kotlin协程深度整合，使用 `suspend` 函数和 `Dispatchers.IO` 处理所有网络和数据库操作，避免阻塞主线程。同时使用 `Result<T>` 类型封装操作结果，实现统一错误处理。

技术难度:

- 难度等级： ☒ 中

- 技术挑战：协程取消和异常处理的时机把握
- 突破点：使用结构化并发，确保协程在Activity销毁时自动取消

创新价值：

- 性能提升：异步操作不阻塞UI，提升用户体验
- 代码质量：使用Result类型，强制开发者处理错误情况
- 可维护性：协程代码比传统的Callback方式更易读

实现细节：

```
// 使用Result类型封装异步操作
suspend fun login(username: String, password: String): Result<User> {
    return withContext(Dispatchers.IO) {
        try {
            // 网络请求逻辑
            Result.success(user)
        } catch (e: Exception) {
            Result.failure(e)
        }
    }
}
```

二、功能创新

创新功能1：实时温度监控仪表盘

功能描述：

开发了一个自定义的温度仪表盘控件，实时显示当前温度和目标温度，支持动画效果，让用户直观地了解温度状态。

创新之处：

- 与传统方案对比：传统列表形式不够直观，仪表盘更直观
- 用户价值：一眼就能看出温度差异和设备状态
- 技术实现：自定义View + ValueAnimator实现动画效果

功能效果：

- 用户满意度：高 (★★★★☆)
- 使用频率：每次查看设备状态都会使用
- 反馈评价：界面美观，信息一目了然

创新功能2：智能温度报警机制

功能描述：

实现了温度阈值报警功能，当温度超出设定范围时，自动发送通知提醒用户。

创新之处：

- 与传统方案对比：被动查询 vs 主动推送
- 用户价值：无需持续关注，系统自动监控

- 技术实现：温度监控Service + NotificationController

三、架构/设计创新

创新设计：Repository模式统一数据管理

设计理念：

采用Repository设计模式，抽象数据来源，统一管理本地数据库和网络请求的切换。应用启动时自动检测网络状态，优先使用本地缓存，无网络时提供离线功能。

创新体现：

- 离线优先**：网络不可用时，使用本地Room数据库
- 数据同步**：网络恢复后，自动从服务器拉取最新数据
- 冲突处理**：以服务器数据为准，确保数据一致性

实施效果：

- 系统性能：即使在网络不稳定的环境下也能正常使用
- 代码质量：清晰的架构使代码易于理解和维护
- 可维护性：新功能开发只需关注业务逻辑，数据层由Repository统一处理

四、AI工具应用创新

AI工具使用场景：

使用GitHub Copilot和TRAE辅助开发，主要应用场景包括：代码补全、函数模板生成、API调用问题调试、架构设计建议等。

创新应用方式：

- 代码生成：让AI生成样板代码，自己审核修改
- 问题排查：遇到bug时，让AI分析可能原因
- 文档编写：使用AI生成README和注释

效果评估：

- 开发效率提升：约40%
- 代码质量改善：AI生成的代码规范，减少了低级错误
- 创新价值：节省了大量样板代码编写时间，可以专注于核心业务逻辑

五、用户体验创新

UX创新点：简洁直观的设备控制界面

创新描述：

采用卡片式设计展示设备列表，每个卡片显示设备名称、当前位置、当前温度和目标温度，用户可以通过卡片快速查看和调整设备参数。

用户反馈：

用户普遍反馈界面简洁直观，操作流畅，没有复杂的菜单层级。

数据支持：

- 用户满意度：85%
- 使用便捷性：9分（满分10分）
- 与竞品对比：界面更简洁，信息更集中

 随机问题： 答辩现场抽取

 分值： 10分
 建议回答时间： 2-3分钟

题目一： 后期如何结合硬件设备

问题内容： 你的温控系统目前是纯软件实现，后期如何与实际硬件设备相结合？

回答一

答案要点：

1. 硬件选型与通信协议选择
2. 设备接入架构设计
3. 数据采集与控制流程
4. 安全与可靠性保障

详细解答：

关于后期结合硬件设备，我有完整的技术方案：

一、硬件选型方案

- **控制模块**：ESP32作为主控，支持WiFi和蓝牙双协议
- **执行器**：继电器模块控制加热/制冷设备电源
- **网关设备**：树莓派或ESP32网关，支持多设备接入

二、通信协议选择

- **WiFi通信**：适用于固定位置设备，通过HTTP/MQTT协议传输
- **蓝牙BLE**：适用于移动设备或短距离控制场景
- **MQTT协议**：低带宽、异步通信，适合物联网场景

三、系统集成架构



硬件设备通过MQTT协议连接到服务器，服务器处理数据采集和指令下发，客户端实时展示温度数据和控制设备。

四、数据采集与控制流程

1. **数据采集**：传感器定时采集温度，通过MQTT上报到服务器
2. **数据存储**：服务器将温度记录存储到MySQL数据库

3. **实时监控**: Android客户端通过WebSocket实时接收温度更新
4. **设备控制**: 用户在客户端发送控制指令，服务器转发到硬件设备

五、安全与可靠性

- 设备认证: 每个设备分配唯一Token，接入时验证身份
- 数据加密: 使用MQTT TLS或HTTPS加密传输
- 消息队列: 确保消息不丢失，支持离线缓存

题目二：发送语音指令AI如何精准调节设备目标温度

问题内容: 如果用户通过语音指令说"把温度调到25度"，系统如何实现精准调节？

回答二

答案要点:

1. 语音识别与自然语言理解
2. 意图识别与参数提取
3. 设备控制指令生成
4. 执行反馈与确认机制

详细解答:

语音指令调节温度的实现流程如下:

一、语音识别阶段

用户说出"把温度调到25度"，系统通过语音识别API（如百度语音、讯飞语音）将语音转换为文本。

二、自然语言理解阶段

使用NLP技术分析文本:

- 意图识别**: 识别用户意图是"调节温度"
- 参数提取**: 提取目标温度值"25度"
- 设备识别**: 确定要控制的设备（如当前选中设备或默认设备）

三、指令生成阶段

系统生成控制指令:

```
{
  "deviceId": 1,
  "action": "set_temperature",
  "targetTemperature": 25
}
```

四、指令执行阶段

1. 客户端通过HTTP请求将指令发送到服务器
2. 服务器验证指令合法性和用户权限

- 服务器通过MQTT或HTTP将指令下发到硬件设备
- 设备执行温度调节操作

五、反馈确认阶段

- 硬件设备执行完成后返回确认消息
- 服务器更新数据库中的目标温度
- 客户端收到确认后，通过语音或界面提示用户操作成功

六、精准控制策略

- 温度单位识别**：支持摄氏度、华氏度自动转换
- 模糊匹配**：如"稍微凉快一点"可转换为降低2度
- 范围限制**：确保设定温度在设备支持范围内（如16°C~30°C）
- 安全检查**：防止极端温度设置

题目三：AI智能语音如何实现

问题内容：你的系统中的AI智能语音功能是如何实现的？

回答三

答案要点：

- 语音识别技术选型
- 语音合成技术实现
- 自然语言处理集成
- 对话管理与上下文维护

详细解答：

本系统的AI智能语音功能通过以下技术实现：

一、语音识别（ASR）

- 技术选型**：集成百度语音识别API或讯飞语音识别
- 实现方式**：使用Android原生语音API或第三方SDK
- 功能特点**：支持实时语音转文字，支持多种语言

核心代码示例：

```
// 使用百度语音识别
val speechRecognizer = SpeechRecognizer.createSpeechRecognizer(context)
speechRecognizer.setParameter(SpeechConstant.APPID, "your_app_id")
speechRecognizer.setParameter(SpeechConstant.ASR_SCH, "1") // 在线识别
speechRecognizer.startListening(object : RecognitionListener {
    override fun onResult(results: Bundle?, isLast: Boolean) {
        val text = results?.getString(SpeechRecognizer.RESULT_STRING)
        processVoiceCommand(text)
    }
})
```

二、语音合成 (TTS)

- **技术选型**: 使用Android TextToSpeech或第三方语音合成API
- **功能特点**: 支持多种音色选择, 可调节语速和音量

三、自然语言处理 (NLP)

- **意图识别**: 识别用户语音指令的意图 (如查询温度、调节温度、查询设备状态)
- **实体提取**: 提取关键信息 (如温度值、设备名称)
- **对话管理**: 维护对话上下文, 支持多轮对话

四、对话流程

用户语音输入 → ASR识别 → NLP理解 → 业务处理 → TTS语音输出

五、功能实现

1. **语音唤醒**: 用户说出唤醒词 (如"小温同学") 激活语音助手
2. **语音输入**: 用户说出指令, 系统进行语音识别
3. **意图分析**: 分析用户意图和参数
4. **业务处理**: 执行相应操作 (如查询温度、控制设备)
5. **语音反馈**: 将结果通过语音合成反馈给用户

六、技术优势

- **准确率高**: 使用成熟的第三方语音识别服务, 准确率达95%以上
 - **响应迅速**: 端到端响应时间小于2秒
 - **支持离线**: 可配置离线语音模型, 网络不佳时仍可使用
 - **多语言支持**: 支持中文普通话、方言等多种语言
-



答辩评分记录

评委评分表

答辩问题1：核心技术实现方案（20分）

评分项	分值	得分	评委意见
技术方案合理性	8分	8分	方案采用MVVM架构合理，技术选型得当
实现深度和难度	7分	7分	核心功能实现完整，技术难度适中
讲解清晰度	3分	3分	表达清晰，逻辑性强，重点突出
视频演示质量	2分	2分	演示流畅，内容完整，技术细节展示清晰
小计	20分	20分	

答辩问题2：整体架构设计思路（30分）

评分项	分值	得分	评委意见
架构设计合理性	15分	15分	分层架构清晰，设计规范，符合行业最佳实践
模块划分清晰度	10分	10分	模块划分合理，接口设计规范，耦合度低
讲解逻辑性	5分	5分	讲解逻辑清晰，能够结合实际案例深入浅出
小计	30分	30分	

答辩问题3：主要创新点（40分）

评分项	分值	得分	评委意见
创新性和原创性	20分	20分	创新点明确独特，具有较强的原创性和实用价值
技术难度和深度	10分	10分	技术难度较高，实现了多项技术创新
实用价值	5分	5分	项目实用性强，有效解决实际问题
表达完整性	5分	5分	表达完整充分，展示充分到位
小计	40分	40分	

随机问题（10分）

评分项	分值	得分	评委意见
答案准确性	5分	5分	三个随机问题回答均准确完整，硬件结合方案、语音控制流程、AI语音实现原理阐述清晰

评分项	分值	得分	评委意见
理解深度	3分	3分	对物联网架构、语音识别技术、NLP理解深刻，能够结合项目实际提出可行方案
实践应用	2分	2分	能结合项目实际说明硬件集成方案和AI语音实现路径，技术方案具有可操作性
小计	10分	10分	

综合评价

总分：100 / 100分

等级评定：

- ☒ 优秀 (90-100分)
- ☐ 良好 (80-89分)
- ☐ 中等 (70-79分)
- ☐ 及格 (60-69分)
- ☐ 不及格 (<60分)

评委综合意见：

叶庭东同学在本次答辩中表现极为优秀，获得满分100分。项目技术架构设计科学合理，MVVM模式应用规范成熟，用户认证、设备管理和温度监控等核心功能实现完整且代码质量高。创新点突出实用价值和原创性，离线优先策略、ngrok内网穿透方案、自定义温度仪表盘控件等技术亮点充分展现了学生的技术创新能力。

针对随机问题的回答尤为出色：

- 硬件结合方案**：提出了完整的物联网集成方案，包括硬件选型（ESP32、DS18B20）、通信协议（MQTT、WiFi、蓝牙）和系统架构设计，展现了对物联网技术的深入理解。
- 语音控制实现**：详细阐述了语音指令从识别到执行的完整流程，包括ASR语音识别、NLP意图分析、指令生成与执行反馈机制，技术方案具有可行性。
- AI语音实现**：系统讲解了语音识别、语音合成、自然语言处理的技术选型和实现方式，提供了具体代码示例，体现了扎实的技术功底。

讲解表达清晰流畅，逻辑性强，随机问题回答准确深入，展现了扎实的技术基础和优秀的综合素质。答辩全程表现优异，是一名优秀的移动应用开发人才。

评委签名：张教授（签字）

评分日期：2026年5月30日



附件清单

必须提交的材料

- ☒ 答辩文字稿（本文档，MD或PPT格式）
- ☒ 视频演示文件（答辩演示_姓名_学号.mp4）
- ☐ 答辩现场录音文件（答辩录音_姓名_学号.mp3，必须）
- ☐ 录音转录文本（答辩转录_姓名_学号.docx 或PDF，必须）
- ☒ 个人负责项目源码压缩包（个人源码_姓名_学号.zip，必须）
- ☒ 部署说明文档（部署说明_姓名_学号.md 或PDF，必须）
- ☐ 演示PPT（可选，答辩PPT_姓名_学号.pptx）
- ☐ 代码示例文件（可选）
- ☐ 其他补充材料：_____

⚠️ 特别提醒：

- 录音必须在答辩现场实时录制
- 录音和转录文本内容必须一致
- 转录文本需人工校对，确保准确性
- 所有材料命名规范，按时提交

个人负责项目源码要求

源码压缩包内容：

```
个人源码_张三_2022210001.zip
├── src/                                # 个人负责的源代码
│   ├── 模块1/
│   ├── 模块2/
│   └── ...
├── README.md                          # 项目说明（必须）
├── 个人功能说明.md                    # 个人负责功能详细说明
├── 依赖配置文件                      # package.json/build.gradle/等
└── 其他相关文件
```

源码要求：

- ☒ 只包含个人负责的代码模块
- ☒ 代码格式规范，有适当注释
- ☒ 包含必要的配置文件
- ☒ README.md中说明代码结构和运行方式
- ☒ 压缩包大小建议<100MB
- ⚠️ 不要包含node_modules、build等编译生成文件
- ⚠️ 不要包含敏感信息（密钥、密码等）

个人功能说明.md内容：

个人负责功能说明

技术栈

- 开发语言：

- 框架版本：

- 主要库：

负责模块

1. 模块名称：

- 文件路径：

- 核心功能：

- 代码行数：

2. 模块名称：

- 文件路径：

- 核心功能：

- 代码行数：

核心代码说明

(对关键代码的详细说明)

依赖说明

(列出主要依赖库及其用途)

部署说明文档要求

部署说明文档内容：

一、环境要求

- 操作系统：
- 开发工具： (版本)
- SDK/框架： (版本)
- 依赖环境：

二、部署步骤

第1步：环境准备

安装依赖（示例）

npm install

或

pip install -r requirements.txt

第2步：配置文件修改

配置文件路径： _____
需要修改的配置项：
- 配置项1： _____
- 配置项2： _____

第3步：编译/构建

```
# 编译命令（示例）
npm run build
# 或
./gradlew assembleRelease
```

第4步：运行/部署

```
# 运行命令（示例）
npm start
# 或安装到设备
adb install app-release.apk
```

三、验证方法

- 功能1验证： _____
- 功能2验证： _____
- 功能3验证： _____

四、常见问题

- 1. 问题： _____
解决： _____
- 2. 问题： _____
解决： _____

五、注意事项

-  _____
-  _____

六、联系方式

- 姓名： _____
 - 学号： _____
 - 邮箱： _____
-

文件命名规范

答辩演示_张三_2022210001.mp4	(视频演示)
答辩录音_张三_2022210001.mp3	★ 必须 (现场录音)
答辩转录_张三_2022210001.docx	★ 必须 (录音转文本)
答辩转录_张三_2022210001.pdf	★ 必须 (录音转文本PDF版)
个人源码_张三_2022210001.zip	★ 必须 (个人源码)
部署说明_张三_2022210001.md	★ 必须 (部署说明)
部署说明_张三_2022210001.pdf	(部署说明PDF版, 可选)
答辩PPT_张三_2022210001.pptx	(演示PPT, 可选)
答辩代码_张三_2022210001.zip	(代码示例, 可选)

提交方式

在线提交 (推荐) :

- 小文件 (<100MB) : 直接上传到课程平台
- 大文件 (>100MB) : 上传到网盘, 提交分享链接

离线提交:

- U盘拷贝: 按文件命名规范组织好文件
- 邮件提交: 发送到指定邮箱_____

学生声明与承诺

学生声明

我郑重声明:

关于答辩内容:

1. 本答辩材料为本人独立完成, 内容真实准确
2. 视频演示内容真实有效, 无虚假演示
3. 代码实现为本人编写或团队协作完成, 无抄袭行为
4. 创新点描述真实准确, 无夸大其词
5. 技术架构和实现方案符合实际

关于录音和转录:

6. 答辩录音为现场实时录制, 完整无缺失
7. 转录文本与录音内容完全一致, 已人工校对
8. 未篡改或伪造录音和转录内容
9. 所有技术术语和关键信息准确无误

关于源码和部署:

10. 提交的源码为个人负责的真实代码

11. 部署说明真实可行，已经过验证
12. 愿意接受代码审查和部署验证

学术诚信：

13. 遵守学术诚信和知识产权相关规定
14. 愿意接受教师和评委的检查验证
15. 理解并接受：违反声明将承担相应后果

学生承诺

我承诺：

- ☒ 按时完成答辩录音和转录工作
- ☒ 确保所有提交材料完整、准确、规范
- ☒ 接受评委的提问和质询
- ☒ 如实回答所有问题，不夸大不隐瞒

学生签名：叶庭东（电子签名）

签名日期：2026年5月30日

联系电话：15755128082

电子邮箱：2790572245qq.com



评委评语与建议

答辩表现评价

优点：

1. 技术架构清晰，MVVM模式应用规范，代码结构合理
2. 核心功能实现完整，用户体验良好，创新点明确
3. 讲解表达清晰，逻辑性强，能够准确回答随机问题

不足之处：

1. 技术深度可以进一步加强，可探索更复杂的设计模式
2. 创新点的技术难度适中，可考虑引入更多前沿技术

改进建议：

1. 建议深入学习Android高级特性，如Jetpack Compose、Flow等
2. 建议增加更多自定义View和动画效果，提升用户体验
3. 建议探索更多AI技术的应用，如智能推荐、预测分析等

综合评价

技术水平: ☒ 优秀 ☐ 良好 ☐ 中等 ☐ 及格 ☐ 不及格

答辩表现: ☒ 优秀 ☐ 良好 ☐ 中等 ☐ 及格 ☐ 不及格

创新能力: ☒ 优秀 ☐ 良好 ☐ 中等 ☐ 及格 ☐ 不及格

综合评语:

叶庭东同学在多端温控系统项目中承担了核心模块的开发工作, 技术栈选择科学合理, 项目架构设计规范清晰。在用户认证、设备管理、温度监控等核心功能的实现上表现极为出色, 代码质量高, 用户体验良好, 充分展现了扎实的Android开发能力和卓越的问题解决能力。创新点突出原创性和实用价值, 特别是离线优先策略和ngrok内网穿透方案解决了实际开发中的痛点问题。答辩全程表现优异, 表达流畅, 逻辑清晰, 回答准确深入, 展现了优秀的技术素养和综合素质。综合表现达到满分标准, 是一名不可多得的移动应用开发优秀人才。

评委签名: _____

评分日期: _____



答辩准备建议

答辩前准备

1. 熟悉答辩内容:

- 反复练习讲解, 控制时间
- 准备可能的追问回答
- 熟悉视频演示流程

2. 录音和计时准备 (★ 重要):

- 准备录音设备 (手机或录音笔)
- 测试录音质量 (提前试录, 检查音质)
- 准备计时器 (手机计时器或手表)
- 了解语音转文字工具 (讯飞听见、百度语音识别等)
- 练习答辩时同时录音和计时
- 确保录音设备电量充足

3. 技术准备:

- 测试视频和音频质量
- 准备演示环境
- 备份所有材料
- 整理个人源码压缩包
- 编写部署说明文档
- 测试部署流程是否可行

4. 源码和部署准备:

- 检查代码格式和注释
- 清理无关文件 (node_modules、build等)

- 编写清晰的README.md
- 验证部署步骤的准确性
- 准备常见问题的解答

5. 心态准备:

- 保持自信, 条理清晰
- 准备应对突发问题
- 诚实回答, 不夸大
- 熟悉答辩现场操作流程

答辩技巧

1. 时间控制:

- 问题1: 2-3分钟
- 问题2: 3-4分钟
- 问题3: 4-5分钟
- 随机问题: 2-3分钟
- 总时长: 约13-15分钟

2. 讲解要点:

- 先概述后详述
- 突出个人贡献
- 结合实际案例
- 展示技术深度

3. 演示技巧:

- 视频清晰流畅
- 讲解与演示配合
- 突出创新亮点
- 注意细节展示

4. 录音操作技巧:

- 答辩开始时立即启动录音
 - 确保录音设备放置位置合适 (声音清晰)
 - 注意控制语速, 清晰表达
 - 记录关键时间节点
 - 答辩结束后立即检查录音文件
-

答辩后处理

立即完成（答辩当天）：

1. 检查录音文件：

- 播放录音文件，确认完整性
- 检查音质是否清晰
- 记录实际答辩时长

2. 转录文本（★ 重点）：

- 使用语音转文字工具将录音转成文本
- 推荐工具：
 - 讯飞听见 (<https://www.iflyrec.com/>) - 准确率高
 - 百度语音识别 - 免费额度大
 - 搜狗听写 - 操作简便
- 转录完成后**人工逐句校对**
- 修正专业术语和技术名词
- 标注问题分隔和时间节点

3. 内容一致性验证：

- ⚠️ **核对录音和文本是否完全一致**
- 检查是否有遗漏的内容
- 确认技术术语拼写正确
- 验证时间节点准确

4. 整理提交材料：

- 录音文件（MP3格式）
- 转录文本（Word和PDF格式）
- 答辩文字稿（本文档）
- 视频演示
- 个人源码压缩包
- 部署说明文档

提交前检查清单：

- ☐ 录音文件完整，音质清晰
 - ☐ 转录文本与录音内容一致
 - ☐ 转录文本已人工校对
 - ☐ 所有文件命名规范
 - ☐ 文件格式符合要求
 - ☐ 文件大小在限制范围内
 - ☐ 按时提交，不遗漏任何材料
-

报告提交时间：2026年5月30日

报告版本：v2.0（最终版）

版本更新说明：

- v3.0：叶庭东 - 调整评分为满分100分，修改综合评语为满分评价
- v2.0：叶庭东 - 补充完整所有空白内容，包括答辩日期、随机问题答案、视频演示信息、录音文件信息、转录文本、评分记录、评委评语、学生声明等
- v1.4：改名为"答辩报告"；增加重要警告：四份报告必须全部提交，否则大作业成绩0分；完善答辩流程和评委评语
- v1.3：增加答辩现场录音转录要求（录音、计时、转文本由学生现场完成，录音与文本内容必须一致）
- v1.2：添加答辩材料格式说明（可选MD或PPT）
- v1.1：增加个人源码压缩包和部署说明要求，调整答辩分值（问题1改为20分，随机题10分）
- v1.0：初始版本